



UNIVERSITY OF
SURREY

Extensive Security Verification of the LoRaWAN Key-Establishment: Insecurities & Patches

Wesemeyer, Stephan; Boureau, Ioana; Smith, Zach; Treharne, Helen

<https://openresearch.surrey.ac.uk/esploro/outputs/conferenceProceeding/Extensive-Security-Verification-of-the-LoRaWAN/99550123702346/filesAndLinks?index=0>

Wesemeyer, S., Boureau, I., Smith, Z., & Treharne, H. (2020). Extensive Security Verification of the LoRaWAN Key-Establishment: Insecurities & Patches. 2020 IEEE European Symposium on Security and Privacy (EuroS&P), 425–444. <https://doi.org/10.1109/EuroSP48549.2020.00034>
Document Version: Author's Accepted Manuscript

Published Version: <https://doi.org/10.1109/EuroSP48549.2020.00034>

Extensive Security Verification of the LoRaWAN Key-Establishment: Insecurities & Patches

Stephan Wesemeyer¹, Ioana Boureanu¹, Zach Smith², Helen Treharne¹

¹ University of Surrey, Surrey Centre for Cyber Security, Guildford, UK

² University of Luxembourg, Esch-sur-Alzette, Luxembourg

Email: s.wesemeyer@surrey.ac.uk, i.boureanu@surrey.ac.uk, zach.smith@uni.lu, h.treharne@surrey.ac.uk

Abstract—LoRaWAN (Low-power Wide-Area Networks) is the main specification for application-level IoT (Internet of Things). The current version, published in October 2017, is LoRaWAN 1.1, with its 1.0 precursor still being the main specification supported by commercial devices such as PyCom LoRa transceivers.

Prior (semi)-formal investigations into the security of the LoRaWAN protocols are scarce, especially for LoRaWAN 1.1. Moreover, amongst these few, the current encodings [4], [9] of LoRaWAN into verification tools unfortunately rely on much-simplified versions of the LoRaWAN protocols, undermining the relevance of the results in practice.

In this paper, we fill in some of these gaps. Whilst we briefly discuss the most recent cryptographic-orientated works [5] that looked at LoRaWAN 1.1, our true focus is on producing formal analyses of the security and correctness of LoRaWAN, mechanised inside automated tools. To this end, we use the state-of-the-art prover, Tamarin. Importantly, our Tamarin models are a faithful and precise rendering of the LoRaWAN specifications. For example, we model the bespoke nonce-generation mechanisms newly introduced in LoRaWAN 1.1, as well as the “classical” but short-domain nonces in LoRaWAN 1.0 and make recommendations regarding these.

Whilst we include small parts on device-commissioning and application-level traffic, we primarily scrutinise the Join Procedure of LoRaWAN, and focus on version 1.1 of the specification, but also include an analysis of LoRaWAN 1.0. To this end, we consider three increasingly strong threat models, resting on a Dolev-Yao attacker acting modulo different requirements made on various channels (e.g., secure/insecure) and the level of trust placed on entities (e.g., honest/corruptible network servers). Importantly, one of these threat models is exactly in line with the LoRaWAN specification, yet it unfortunately still leads to attacks.

In response to the exhibited attacks, we propose a minimal patch of the LoRaWAN 1.1 Join Procedure, which is as backwards-compatible as possible with the current version. We analyse and prove this patch secure in the strongest threat model mentioned above.

This work has been responsibly disclosed to the LoRa Alliance, and we are liaising with the Security Working Group of the LoRa Alliance, in order to improve the clarity of the LoRaWAN 1.1 specifications in light of our findings, but also by using formal analysis as part of a feedback-loop of future and current specification writing.

1. Introduction

The Internet of Things is growing at a phenomenal rate, with billions of active devices. The implementation of connected sensors on small, embedded chips allows for data collection on an unprecedented scale.

The LoRa Alliance represents a collaboration between over 500 companies, producing a set of standards for IoT devices. The most notable of their contributions is the LoRaWAN specification for **Long Range** devices over a Wide Area Network (WAN). Over 80 million connected devices¹ follow the LoRaWAN specification, meaning that its security is of key interest. In October 2017, the LoRaWAN specification was updated from version 1.02 to version 1.1, containing significant updates to several sections. However, because many IoT devices run specialised hardware, the upgrade path has been very slow, with many devices remaining un-updated.

As such, both the LoRaWAN 1.0 and the LoRaWAN 1.1 specifications remain of interest to many communities. In this paper, we are interested in formally analysing their security and, particularly, the security of the LoRaWAN Join Procedure – in which a LoRaWAN Device uses a LoRaWAN Join Server to establish channel-keys used to communicate securely with a LoRaWAN Application Server. Although certain security concerns regarding the LoRaWAN Join have been raised in academic papers [1], in general, there has not been significant engagement of the LoRa Alliance with the academic community of formal security-analysis. This type of engagement is arguably desirable, considering – for instance – the success of such collaborations in recent years, such as the formal modelling/analysis of TLS 1.3 [6] before its official release.

Related Work. There are several published discussions of vulnerabilities of LoRaWAN 1.0 Join [1], [9], using multiple different approaches. On the one hand, one of these [9] is semi-formal (using symbolic verification), yet it is arguably not a faithful analysis of LoRaWAN 1.0, as their modelling is over-simplified. On the other hand, [1] carries out a “pen-and-paper” analysis of the LoRaWAN 1.0 Join (i.e., not using formal methods), yet signals multiple security issues.

If the LoRaWAN 1.0 Join is not widely scrutinised, there are even fewer assessments of the security of LoRaWAN 1.1 Join [4], [5]. The frameworks used in these

1. https://lora-alliance.org/sites/default/files/2019-02/LORAWAN_LIVE_MASTER_DECK_FINAL_FOR_RELEASE.pdf

works do not always faithfully capture the additional details added in the latest version of the specification. Concretely, neither [4] nor [5] fully consider that the 1.1 Join is a four party authenticated key-exchange (AKE) protocol. In the case of [4], not only is it the case that the analysis is treated at the two-party AKE level and that Join Server is not modelled at all, but the choices of modelling abstractions remove important details and their specification files found online are incomplete. At the other end of the spectrum, Canard *et al.* [5] provide a formal, rigorous cryptographic model. However, they build on the 3ACCE model [2] (which is for proxied 3-party AKEs), and fail to fully address the fourth party (i.e., the Application Server is not fully modelled).

Objectives. We aim to analyse the AKE protocols in the LoRaWAN 1.0 and 1.1 Join Procedures, as faithfully as possible, considering several threat models which accurately portray the real-world possibilities.

To do this, we make use of the Dolev-Yao [7] attacker: a man-in-the-middle who can intercept, redirect and modify any message, but is bound by the assumption of perfect cryptography. In other words, we operate in the space of symbolic verification/analysis of security protocols.

Contributions. The main contributions we present are as follows:

- We further the analysis of the LoRaWAN 1.0 Join, creating automated, tool-assisted proofs of the existence of attacks therein that had previously been demonstrated only by hand in [1].
- We present faithful symbolic analyses, far beyond pre-existing efforts of this type [4], [9], of the LoRaWAN 1.1 Join, *closely following its specification*. We analyse the LoRaWAN 1.1 Join under multiple threat models, including the one declared by the LoRa Alliance. In these series of model, we also include the case where the Network Server (which acts as a proxy) may be partially dishonest. To our knowledge, this is the *first full symbolic analysis of the LoRaWAN 1.1 Join*. Using this analysis, we demonstrate several security violations, including those in a model which closely follows that of the specification. Following from this, we discuss the shortcomings, inexactitudes and implications of the current specification for LoRaWAN 1.1 Join.
- Drawing on the above, we propose a new design of LoRaWAN 1.1 Join, which we call *LoRaWAN 1.1 LORA 3-AKA⁺*. This is as backwards-compatible as possible with the current LoRaWAN 1.1 Join, and we prove using the Tamarin proving tool that LORA 3-AKA⁺ is secure even in our strongest threat model.

Structure. The rest of the paper is structured as follows. In Section 2 we describe the Join procedure in the LoRaWAN 1.0 and 1.1 specifications, and give background on the Tamarin prover tool [15]). In Section 3 we introduce our threat models, as well as establishing our desired security goals. Section 4 contains a summary of the modelling and analysis of the two Join procedures. In Section 5 we present LORA 3-AKA⁺, our proposal for a new Join procedure which addresses the vulnerabilities identified. We then give the full results of our analysis in Section 6, which highlights the consequences of different

modelling assumptions and threat models. In Section 7, we report of the discussions we had with the LoRa Alliance regarding to this work and its ramifications. Finally, we conclude in Section 8.

2. Background

In this section, we present the background and foundations to this work.

2.1. The Join Procedure in LoRaWAN 1.1

Protocol Description. The LoRaWAN 1.1 “Join procedure” is a four-party protocol [13], involving an *end device ED*, a *Join Server JS*, a *Network Server NS* and an *Application Server AS*. The protocol relies primarily on the fact that *ED* and *JS* share two² long-term symmetric keys: AppKey and NwkKey. The aim of the protocol is to use these long-term keys to establish a set of session keys for use in the record layer.

In Figure 1, we depict the Join Procedure in LoRaWAN 1.1, where –for simplicity– we omit certain low-level details, such as certain identifiers, message padding, and headers. Appendix B contains an enlarged figure with more details. The protocol proceeds as follows:

- 1) *ED* sends to *JS* a “Join Request” formed of a nonce DevNonce, relevant identifiers, along with a MAC of this information using the long-term key NwkKey.
- 2) This request is received by *NS*, who forwards it as-is (apart from network-level headers) to *JS*.
- 3) Upon successful verification of the MAC, *JS* produces a “Join Answer”, which includes a “Join Accept”, and a number of session keys. The crucial session keys are AppSKey and NwkSKey. We will discuss these keys’ generation later.
 - To create the “Join Accept”, *JS* generates a new nonce JoinNonce, which it MACs and encrypts using the long-term keys shared with *ED*.
 - In the “Join Answer”, the AppSKey is encrypted with a long-term key that *JS* shares with *AS*.
 - The “Join Answer” is sent by *JS* to *NS*.
- 4) The Network Server retains NwkSKey and forwards only the “Join Accept” part. From the “Join Accept”, the device retrieves JoinNonce, and is then able to re-compute the session keys.
- 5–6) Later, in the first encrypted application-level message sent by *ED*, the application server receives either the encrypted AppSKey from the *NS*, or a session-id with which to obtain the AppSKey from the *JS*. The *NS* forwards this information along with the first application-level message. This will be further discussed later.

Keys Generated During the Join Procedure. During the Join Procedure, the Join Server (and later, the End Device) generates a set of session keys. The most significant of these are named AppSKey and NwkSKey³. These sessions keys are produced by encrypting with AES-128

2. There is a third, long-term shared key called JSIntKey.

3. There are two additional MAC keys for *NS* and *ED*, namely, SNwkSIntKey and FNwkSIntKey. Our formal models capture all session keys.

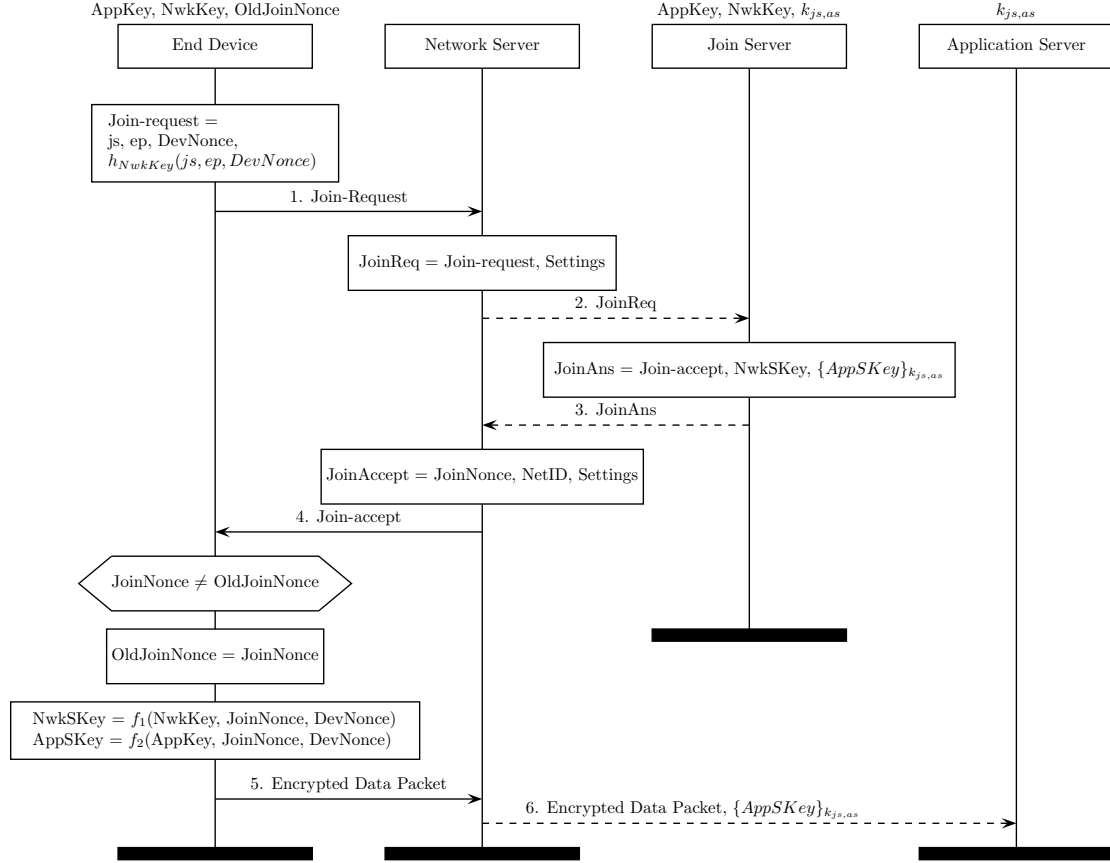


Figure 1: LoRaWAN 1.1 Join Procedure – Simplified View. Secure channels are displayed with dashed lines --->

using the long-term keys AppKey and NwkKey, respectively. The following data is used in the key generation: DevNonce, JoinNonce and *JS*'s identifier JoinEUI, along with specific message tagging⁴ (e.g., 0x02, 0x04, respectively). Each session key is used at the record layer for encrypting/decrypting application messages⁵: NwkSKey is to be used by *NS* and *ED*, and AppSKey by *AS* and *ED*.

2.2. LoRaWAN 1.1. Join Procedure Observations

There are a few other aspects of the LoRaWAN 1.1 "Join Procedure" worth mentioning, which we detail below.

LoRaWAN 1.1 Channel Requirements. During the Join Procedure, as per the LoRaWAN 1.1 current specifications (i.e., 2017 version), the security of channels is assumed to be as follows: insecure channel between *ED* and *NS*, insecure channel between *NS* and *AS*, a secure channel between *NS* and *JS*. Upon the end of this AKE, the intended channels are as follows: (a) confidential, authenticated and integral channel between *ED* and *NS* (obtained via encryption and MAC-ings); (b) insecure channel⁶ between *NS* and *AS*, on which a

correct behaviour is one where *NS* is proxying AppSKey-encrypted messages between *ED* and *AS*. There is also a secure channel between *AS* and *JS*, which can optionally be used to deliver the AppSKey.

Note the insecure channels between *NS* and *AS*. More precisely, the main LoRaWAN 1.1 specifications and the backend specifications as they stand (in 2017), make no explicit, clear or mandatory requirement of integrity or of confidentiality with respect to the channel between *NS* and *AS* during the Join or thereafter, at the application level.

Bespoke Nonce-generation in LoRaWAN 1.1. In LoRaWAN 1.1, DevNonce is not sampled uniformly at random over its domain. Instead it is produced as a strictly-increasing counter. *NS* keeps a tally of DevNonces per device, and checks that it is increasing (see page 52 of the specifications). Similarly, the JoinNonce (which is 24 bits long) is produced by the increasing a counter, kept up-to-date for each device.

Alternative AppSKey-deliveries in LoRaWAN 1.1. There are in fact two ways in which AppSKey can be delivered to *AS*: (1) by the *NS* who would forward it with the first application message (which is shown on Fig 1); (2) by the *JS*, upon request. For the latter, the *JS* needs to have generated a SessionKeyID and that be sent alongside the encrypted AppSKey in the Join Answer message, to the *NS*. If a SessionKeyID is given, the *NS* can forward this alongside the first data packet. In order to retrieve the encryption key, *AS* contacts *JS*, who delivers the corresponding AppSKey.

4. To not overload notation with different AES keys and different padding, on Fig. 1, we simply write NwkSKey is calculated using a function f_1 and AppSKey is calculated using a function f_2 .

5. LoRa record-layer messages are not encrypted directly with the AppSKey, instead there is a bespoke KDF which uses AppSKey.

6. Concretely, line 1437 of the LoRaWAN 1.1 specifications mentions that, a-priori, there is no secure channel between *NS* and *AS*.

Figure 11, in the appendices, shows this second means of AppSKey delivery.

The specification does not indicate which approach is preferred. In this second case, *AS* cannot immediately decrypt messages sent from *ED*.

In both cases, it is not clear from the specifications whether *AS* is to check if the first data-packet/application-level message does indeed decrypt correctly with the delivered AppSKey and how the protocol continues in case of failure.

2.3. The Join Procedure in LoRaWAN 1.0

Now, we explain the LoRaWAN 1.0 Join procedure protocol. We use as a reference the LoRaWAN 1.0.2 version of the Join Procedure [14]). We focus on highlighting the differences with the LoRaWAN 1.1 Join relevant to the work herein.

2.4. LoRaWAN 1.0 Join vs. LoRaWAN 1.1 Join

Compared to the LoRaWAN 1.1 Join, the LoRaWAN 1.0 Join considers only three active roles, *End Devices (ED)*, a *Network Server (NS)*, and the backend *Application Server (AS)*: i.e., there is no Join Server. The role played by the Join Server in the LoRaWAN 1.1 Join is encapsulated in the LoRaWAN 1.0 Join into the Network Server logic. Also, there are fewer shared long-term keys used in the AKE protocol. In the description below, we denote the singular long-term shared key as AppKey.

Much like in the case of 1.1, the Application Server is not active during the Join protocol, instead receiving the derived AppSKey from the Network Server.

Unlike in the LoRaWAN 1.1 Join, in the 1.0 Join the DevNonce and JoinNonce are sampled uniformly at random over their domain. Yet, DevNonce is only 16-bit long, leading to a birthday paradox-style attack for LoRaWAN 1.0 (i.e., high chance of repetitions over sessions) [1].

2.5. The Join Procedure in LoRaWAN 1.0

Figure 2 shows the messages exchanged in the LoRaWAN 1.0 Join.

The protocol proceeds as follows:

- 1) The End Device sends a “Join Request” message, consisting of a nonce DevNonce along with identifiers for *ED* and *AS*. The message is MAC-ed using the long term key AppKey.
- 2) After checking the freshness of DevNonce⁷, *NS* replies with a “Join Accept” message. This message contains identifiers for *NS*, a fresh nonce AppNonce, and a collection of parameters to be used for further communication. This message is encrypted using AppKey⁸, and a MAC of the plaintext is sent back alongside this encryption.
- 3) *ED* and *NS* can now calculate the session keys NwkSKey and AppSKey, as AES-128 encryptions under AppKey of AppNonce and DevNonce together with specific tagging (e.g., 0x01, 0x02). To make the figure more readable, we write f_1 and f_2 respectively instead of $\text{aes}(\text{AppKey}; 0x01 || \dots)$, $\text{aes}(\text{AppKey}; 0x02 || \dots)$.

2.6. The Tamarin Prover

The Tamarin prover [12] is a popular verification tool that supports symbolic analysis [3] of security protocols in the Dolev-Yao model [7] in the unbounded setting.

Tamarin models are transition systems over a multi-sorted term algebra, operating on the semantics of multiset rewriting logic [8]. Security properties to be analysed are expressed in a guarded first-order logic quantifying over atomic facts declared in the model. For the analysis that we are interested in (i.e., that of trace properties), Tamarin performs a backwards reachability analysis: beginning

7. “For each end-device, the network server keeps track of a certain number of DevNonce values used by the end-device in the past, and ignores join requests with any of these DevNonce values from that end-device.” [14]

8. Technically, the message is *decrypted* using this key, so that the end device needs only implement the primitive for encryption

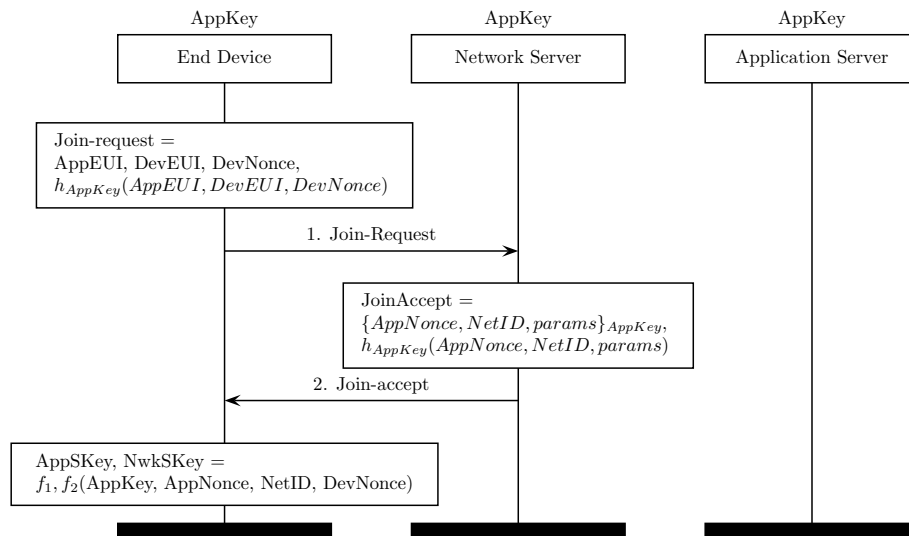


Figure 2: LoRaWAN 1.02 Join Procedure

from a state representing a counterexample to a security claim, it then attempts to build a trace in backwards steps to the empty state. The existence of such a trace represents a violation of the desired security goal. Generally, Tamarin is used as an interactive tool, where the user guides the proof search.

Tamarin Oracles. Tamarin supports various heuristics to cover the search-space yielded by the constraint-solving problem underlying the analysis. These heuristics, similarly to user interaction, determine which rules should be prioritised during the search. To this end, the user can rank different rules: higher ranked rules are triggered with priority. Such a set of rule rankings yields a bespoke search heuristic, named an “oracle”, which can be passed to Tamarin [15]. These oracles only influence termination and proof-search, not the result, such that soundness is preserved.

3. Threat Models & Security Properties

3.1. Threat Models

Balance of Trust & Security in the LoRaWAN 1.1 Specifications. We consider three threat models, which we name $\mathcal{M}_{AS-NS-Secure}$, $\mathcal{M}_{LoRa1.1Spec}$, and $\mathcal{M}_{NS-weakCorrupt}$. The intent of these models is to balance, as well as probe, the trust assumptions and the security requirements made by the LoRaWAN specifications. Concretely, we differentiate these models based on the amount of trust placed in the Network Server, as well as the security of channels between NS and the backend.

		Channel Security	
		High	Low
NS trust	High	$\mathcal{M}_{AS-NS-Secure}$	$\mathcal{M}_{LoRa1.1Spec}$
	Low	—	$\mathcal{M}_{NS-weakCorrupt}$

$\mathcal{M}_{LoRa1.1Spec}$ is the threat model exactly implied by the LoRaWAN 1.1 specifications for the Join Procedure. In this case, there is a high level of trust on NS . More specifically, as per the specifications, we consider all parties be honest. As per the specs, we model the channels between NS and JS and between AS and JS as secure, but the channel between NS and AS as insecure, i.e., as the specifications do not require it be secure.

$\mathcal{M}_{AS-NS-Secure}$ has the same level of trust as $\mathcal{M}_{LoRa1.1Spec}$, i.e., the NS and all other parties are considered honest. However, for $\mathcal{M}_{AS-NS-Secure}$, we consider the channel between NS and AS to be secure. As a result, $\mathcal{M}_{AS-NS-Secure}$ is a weaker threat model than $\mathcal{M}_{LoRa1.1Spec}$. Security analysis in this model is arguably adequate, as a proprietary AS might be commissioned with specific security measures (i.e., authentication, confidentiality and integrity) in its communication with the NS .

$\mathcal{M}_{NS-weakCorrupt}$ is a stronger threat model than $\mathcal{M}_{LoRa1.1Spec}$. Concretely, $\mathcal{M}_{NS-weakCorrupt}$ is in fact the same as the $\mathcal{M}_{LoRa1.1Spec}$ model except that the channel between NS and JS has become insecure. In particular, in $\mathcal{M}_{NS-weakCorrupt}$, the NS behaves as per its specification, and the attacker has extra powers only in that it has compromised the security of the channel between the NS and

the JS . Arguably, this is a weak form of compromising the NS . That is, the model $\mathcal{M}_{NS-weakCorrupt}$ does not take away this entire trust assumption in the LoRaWAN 1.1 Join (i.e., NS does continue to behave follow the protocol); instead, $\mathcal{M}_{NS-weakCorrupt}$ probes the implications of even the smallest compromise of this assumption.

In our formal modelling (in Tamarin), insecure channels are implicitly public and thus accessible to the Dolev-Yao attacker for active manipulation.

So, the $\mathcal{M}_{LoRa1.1Spec}$ model is the baseline (in line with the trust and security of the LoRaWAN specifications), $\mathcal{M}_{NS-weakCorrupt}$ is a weakening of trust assumptions in $\mathcal{M}_{LoRa1.1Spec}$, whereas $\mathcal{M}_{AS-NS-Secure}$ is a strengthening of the secure-setup requirements in $\mathcal{M}_{LoRa1.1Spec}$.

Threat Models for LoRaWAN 1.0. Note that whilst all the above models apply to the LoRaWAN 1.1 Join, they do not apply to the LoRaWAN 1.0 Join. Concretely, $\mathcal{M}_{NS-weakCorrupt}$ rests on the security of the channel between NS and JS , but this channel is not present in the LoRaWAN 1.0 Join (as JS does not exist therein); and in the 1.0 Join, the AS is completely inactive, so $\mathcal{M}_{LoRa1.1Spec}$ and $\mathcal{M}_{AS-NS-Secure}$ do not apply.

3.2. Properties Analysed

For our verification, we encode various agreement and synchronisation properties [11]. Such goals are commonplace for AKE protocols: e.g., weak agreement, non-injective agreement, injective agreement, synchronisation and secrecy of the established key (key secrecy, for short). The main difference is that, for LoRaWAN 1.1, these are extended to encompass not the standard two parties, but rather three or four parties, as demanded by the LoRaWAN Join procedures.

As we give details of our models, which themselves make the properties more specific, we further explain the precise nature of certain formulations of our properties.

4. Security Analysis of the LoRaWAN Joins

In this section, we apply our threat models and examine the security of the LoRaWAN 1.0 and 1.1 Join Procedures. We give an overview of our Tamarin models and summarise the most important observations and their consequences.

All of our Tamarin models can be found via the website <https://loraverif.github.io/>.

4.1. Modelling of LoRaWAN 1.0 Join

We first go over our analysis of the LoRaWAN 1.0 Join procedure. Note that for this protocol we consider only the $\mathcal{M}_{LoRa1.1Spec}$ threat model. This is because there is no Join Server, changing the assumptions on secure channels.

We also do not model confusion freeness, as in this case it is subsumed entirely by the synchronisation goal, `sync_ED_NS`, defined in the following subsection.

Comprehensively & Closely Modelling the LoRaWAN 1.0 Specs in Tamarin. Figure 2, presented earlier, shows a simplified version of the LoRaWAN 1.0 join procedure. However, our Tamarin model follows the

specification much more closely. Appendix A contains an example of the level of detail captured – which includes padding and optional parameters.

Nonce Freshness & Short Nonces. The specification states that *NS* should keep a tally of nonces sent by *ED*, performing necessary freshness checks. Our model fully captures this.

We also simulate the fact that nonces are only over generated small domains ($\text{DevNonce} \in \{0, 1\}^{16}$ and $\text{JoinNonce} \in \{0, 1\}^{24}$). As detailed in the next paragraph, we devise a manner of producing nonces in a cyclic manner. This clearly under-approximates true nonce generation (which would be done by simply using a “Fresh” sort), however it encapsulates the fact that collisions of nonces are likely. It is arguably unusual for symbolic methods to encode this fact, however LoRaWAN 1.0 insecurities stemming from these collisions are known [1]. So, our model aims to see if we can find attacks based on this same short-nonces shortcoming.

Our modelling approach in this case is to encode the set of possible values as constant terms (i.e. ‘1’, ‘2’, ...) rather than abstract fresh terms. This set of values is encoded into the specification of *ED*. When a value is to be chosen, we take advantage of Tamarin’s multiset builtin, which permits nondeterministic selection of an element from a multiset. To verify the soundness of this “small-domain” encoding, we prove two lemmas:

- (a) `two_join_requests_distinct_nonces_device`, stating a device can always send two (or more) Join Requests with different nonces. This ensures that our encoding does not restrict the behaviour too coarsely;
- (b) `two_join_requests_same_nonce_device`, stating that a device can send two Join Requests with the same nonce, demonstrating that our modelling can indeed capture the shortcomings of short-nonce repetition.

4.2. Analysis of LoRaWAN 1.0 Join

We first present the main security properties we encode and verify.

Key-Agreement Properties. Firstly, we focus on standard agreement properties for AKE protocols, as semi-introduced by Lowe [10]. To this end, we encode the following properties with corresponding intuitions:

- (1) *weak agreement* (`wa_ED_NS`) – whenever a device *ED* has completed the Join Procedure with a network server *NS*, then some execution of *NS* has “recorded” to have run the Join with said *ED*;
- (2) *non-injective agreement* (`nia_ED_NS`) – whenever a device *ED* has completed the Join Procedure with a Network Server *NS*, and *ED*’s transcript contains certain messages, then some execution of *NS* has “recorded” to have run the Join with said *ED* with the same said messages;
- (3) *injective agreement* (`ia_ED_NS`) – whenever a device *ED* has completed an execution *d* of the Join Procedure with a network server *NS* and *ED*’s transcript for execution *d* contains certain messages, then there is a unique execution *n* of *NS*, which records a Join Procedure with said *ED*, and the same said messages that appear in *ED*’s execution.

- (4) *synchronisation* (`sync_ED_NS`) – in any full Join execution there exists an *ED* and a *NS* who both have matching views of the transcript of the execution.
- (5) *session-key secrecy* (`key_secretcy`) – if a secret key *x* is established at timepoint *i*, then either the adversary does not know *x*, or the agent who established said key has been compromised beforehand.

Freshness Failure. The main specification encoded for checking freshness and identifying potential replay attacks is the following fifth property:

- (6) *replay-attack existence* (`replay`) – there can exist two Join executions, *i* and *j*, in which $\text{DevNonce}_i = \text{DevNonce}_j$ and $\text{AppNonce}_i = \text{AppNonce}_j$ (i.e., in which both *ED*’s nonce and *NS*’ nonce repeat themselves).

Analysis Results for LoRaWAN 1.0. We now present some details and the results of our analysis for the properties above, performed using the Tamarin prover tool. To carry out the verification, we created an oracle to improve the automatic navigation of the search-space.

The results show that 4 out of the 6 properties examined for agreement and synchronisation in LoRaWAN 1.0 Join are violated. We provide a summary in the table below.

Property	Result
(1) weak agreement (<code>wa_ED_NS</code>)	holds
(2) non-injective agreement (<code>nia_ED_NS</code>)	falsified
(3) injective agreement (<code>ia_ED_NS</code>)	falsified
(4) synchronisation (<code>sync_ED_NS</code>)	falsified
(5) key secrecy (<code>key_secretcy</code>)	holds
(6) replay-attack resistance (<code>replay</code>)	falsified

Table 1: Verification Results of the LoRaWAN 1.0 Join

We now go over some of the results contained in Table 1. Further execution details are given in Section 6.

Interpretation of Attacks. We note that the second and fourth failings, of non-injective agreement and synchronisation respectively, come down to equivalent traces. This trace shows that the main crux of the failure is that *DevNonce* is not used in the LoRaWAN 1.0 Join Response. We note that this behaviour is rectified in the LoRaWAN 1.1 Join.

The fact that replay-attack resistance fails indicates that even with checks by the *NS* on the freshness of *DevNonce*, the protocol does fail to catch replay attacks. This is a result of the small size of the set of possible nonces. Namely, the trace shows that it is possible for there to be a session *i* in which the attacker can replay an old *DevNonce* from a session *j* and, in session *i*, the *NS*’ *AppNonce* also coincides with the *AppNonce* in session *j*. As a result, the *AppSKey* established in each of two traces is the same. Given how the encryption of the record-layer messages works in LoRaWAN 1.0, this means that the attacker can retrieve elements of the plaintext of the record-layer messages from the two sessions *i* and *j*. Note that the replay attack we exhibit by this was shown by hand in 2018 [1]; however, *this is the first time this replay was automatically found by a protocol-verification tool.*

As well as verifying the soundness of our “small-domain” nonce approach, we also verified that this mod-

elling is complete — that the attacker cannot always perform this replay attack. To this end, we prove two additional lemmas, which demonstrate that there are also traces in which only one nonce (be it the DevNonce or the AppNonce) repeats itself.

4.3. Modelling of LoRaWAN 1.1 Join

Comprehensively & Closely Modelling the LoRaWAN 1.1 Specifications in Tamarin. We follow the specifications of the LoRaWAN 1.1 Join closely, encoding in Tamarin levels of details far beyond the simplified description given in Figure 1. For instance, the Tamarin code-snippet shown in Figure 3 shows part of the specification of the JoinServer, which contains details about sessions keys, padding, and counters.

In order to justify the choices made in our modelling, our Tamarin files contain references to the line/page of the LoRaWAN specifications which justify our choice of encoding.

Modelling the Join & Data Packets in the LoRaWAN 1.1. The LoRaWAN 1.1 Join Procedure finishes before the first data packet is sent (i.e., the keys are established before this packet is sent). However, as explained in Subsection 3.2, we wish to verify key-agreement properties with respect to all four parties involved, but *AS* receives the AppSKey only when first data packet is delivered (or shortly afterwards, depending on the delivery method). To this end, we model the full Join Procedure, plus the first data packet.

Variations of the LoRaWAN 1.1 Specifications Modelled. The specification contains no clear requirement on the *AS* being able to decrypt the first data-packet sent to it. To this end, we model two variations of the LoRaWAN 1.1 Join:

- The “*Desync-Model*”, which encodes the LoRaWAN 1.1 specification precisely (where the *AS* will not end in “error” if it receives the first application message malformed);
- The “*Sync-Model*”, which encodes a slight tightening of the LoRaWAN 1.1 specifications, whereby the *AS* will end in “error” if it receives a malformed first application message.

For each such “sync”/“desync” setting, we encode the two cases discussed in Subsection 2.2 as to how *AS* receives the AppSKey. This yields:

- “*AppSKey-from-NS*”, which encodes that the *NS* delivers the (JS-encrypted) AppSKey directly to the *AS*, with the first application-level message (as per Figure 1);
- “*AppSKey-from-JS*”, which encodes that the *JS* delivers the AppSKey to the *AS*, upon the latter’s request yielded after the first application-level message arrived at the *AS* (as per Figure 11).

Encoding Different Threat Models. For both the *Desync-Model* and the *Sync-Model*, as well as for both the *Appskey-from-NS* and *Appskey-from-JS* variations, we create four different Tamarin files, each encapsulating one of the three threat models $\mathcal{M}_{\text{LoRa1.1Spec}}$, $\mathcal{M}_{\text{NS-weakCorrupt}}$ and $\mathcal{M}_{\text{AS-NS-Secure}}$, presented in Subsection 3.1.

Consequently, to capture all these different scenarios, we include a total of 12 Tamarin files for LoRaWAN 1.1. Note that this is mainly done for convenience, with the vast majority of the Tamarin code shared between models, and only minor changes in key locations. This separation allows us to pinpoint which “configurations” of LoRaWAN 1.1 lead to attacks.

Restrictions. In order to improve the tractability of our analysis in the most severe threat model,

```
rule JoinServer_Receive_JoinRequest_Generate_Response_with_Key:
let
...
//verify tau_c
tau_c_dash=<'MHDR', $JoinEUI, $DevEUI, DevNonce>

//generate response - assuming OptNeg is set, ie we are using v1.1
ctr_JS=ctr_JS_in+'2' //counter for nonces
JoinNonce=Nonce(<$DevEUI, ctr_JS>) //nonce -- device specific; counter-based
DevAddr=Nonce('rnd32DevAddr') //random device address assigned by the Join Server

//compute various keys
pad16='pad_with_0s'
//the key used for the mac during the initial Join-Accept answer
JSIntKey=SessionKey(senc(<'0x06', $DevEUI, pad16>, NwkKey))
//the key used for the Join-Accept triggered by a Rejoin-Request
//JSEncKey=senc(<'0x05', DevEUI, pad16>, NwkKey) not needed atm

//compute various network session keys
//Forwarding Network session integrity key (p55, l. 1604)
FNwkSIntKey=SessionKey(senc(<'0x01', JoinNonce, $JoinEUI, DevNonce, pad16>, NwkKey))
//Serving Network session integrity key (p55, l. 1605)
SNwkSIntKey=SessionKey(senc(<'0x03', JoinNonce, $JoinEUI, DevNonce, pad16>, NwkKey))
//Network session encryption key (pp55, l. 1606)
NwkSEncKey=SessionKey(senc(<'0x04', JoinNonce, $JoinEUI, DevNonce, pad16>, NwkKey))
//compute the secret Application Server Key (pp55, l. 1601)
AppSKey=SessionKey(senc(<'0x02', JoinNonce, $JoinEUI, DevNonce, pad16>, AppKey))
...
```

Figure 3: Code Snippet of Tamarin rule for LoRaWAN 1.1 Join Procedure

$\mathcal{M}_{\text{NS-weakCorrupt}}$, we include a series of restrictions on the maximum number of Join Requests and the maximum number of Join Responses. This means that, in this strongest model and for this property, we look to see if there is any attack within a maximum number of spanned sessions for one given Device and one given Join Server. However, we do not bound the attacker’s powers in other ways, such as number of nonces or messages. Concretely, for the LoRaWAN 1.1 Join in the $\mathcal{M}_{\text{NS-weakCorrupt}}$ model, proofs were carried out with a maximum number of Join Responses and Requests set to 4 and 3 respectively. These values were chosen with the tradeoff between computational cost and faithfulness of the model in mind. However, for exhibiting an attack, we found counterexamples of the properties in $\mathcal{M}_{\text{NS-weakCorrupt}}$ even with this set to 2.

Weakly-typed Models vs. Source Lemmas. Specifically for the case of proving properties on the $\mathcal{M}_{\text{NS-weakCorrupt}}$ models, we wrote several Tamarin source lemmas (see [15]). However, these proved ineffective. Instead, we achieved better performance by weakly-typing⁹ the models. Nonetheless, we did this in a parsimonious way, i.e., we only declared: (a) a specific sort Nonce as a subsort of Fresh, to exclude typing-attacks on nonces; (b) a specific format for data-packages after the Join finished, to stop injection of *any* possible term therein. Arguably, these two weak-typings are realistic and cannot exclude mainstream key-agreement attacks. These together with the aforementioned restrictions allowed us to disprove (different versions) of confusion freeness in the strongest attacker model $\mathcal{M}_{\text{NS-weakCorrupt}}$.

Counter-based Nonces. As described in Subsection 2.2, the LoRaWAN 1.1 specifications replace nonces with counter-based bitstrings both on the ED and on the JS , which the latter having device-specific counters. We model this in Tamarin, using the multiset builtin. Counters are modelled as multisets over a domain of a single element (i.e. the counter can have values ‘1’, ‘1’+‘1’...), incrementing on each use. Both ED and JS record the last accepted value of their partner’s counters, and will only accept a message if the counter is increasing.

4.4. Security Goals of LoRaWAN 1.1 Join

We now present the main security properties we encode and verify.

Firstly, to encode equivalent properties as in the LoRaWAN 1.0 Join, we look at agreement properties between ED and JS . To this end, recall that the role of NS in the LoRaWAN 1.0 Join is assumed in LoRaWAN 1.1 by JS . So, with the equivalent meaning as those presented in Subsection 4.2, we encode:

- (1) *key secrecy for the NwkSKey and AppSKey* (key_secrecy)
- (2) *weak agreement between ED and JS* (wa_ED_JS)
- (3) *non-injective agreement between ED and JS* (nia_ED_JS)
- (4) *injective agreement between ED and JS* (ia_ED_JS)

⁹ Such techniques have been used before in Tamarin-verification when large systems (such as TLS1.3) were encoded and analysed [6].

Secondly, we move to encoding AKE requirements linked specifically to the LoRaWAN 1.1 Join, i.e., between the Devices and the Application Server. We specify the following lemmas:

- (4) *weak agreement between ED and AS* (wa_ED_AS) – which encodes that whenever a device ED has completed the LoRaWAN 1.1 Join Procedure allegedly with AS then some execution of AS has “recorded” running with ED .

We note, once again, that (in our models and in the LoRaWAN 1.1 specification), an AS “ascertains” that it is communicating with a specific ED only when said AS receives the first (encrypted) data packet and the corresponding AppSKey to decrypt it, as –beforehand– the AS is not alive during the actual key-establishment run by the ED and JS (via NS). Recall that, to capture error-handling if the packet-decoding by the AS fails, we encode the two variations “Desync-Model” and “Sync-Model” of the LoRaWAN specifications. In both of these models, we look at the synchronisation sync_ED_AS property below:

- (5) *synchronisation between ED and AS* (sync_ED_AS) – which checks if there is an execution in which ED sends a data-packet, yet the data-packet received by AS cannot be decoded.

Moreover, we also encode agreement properties similar to sync_ED_AS but slightly stronger. Namely, we check 2-party and 3-party agreement properties with respect to data packets and/or AppSKeys, to check not only if the AS is (un)able to decode a data-packet (as per sync_ED_AS), but also if an AS could potentially be adversarially confused as to which data-packet and/or AppSKey is associated with which ED . We call these agreement properties **confusion freeness** and describe several of their flavours below.

- (6) *two-party confusion-freeness w.r.t. packets* (CF_ED_AS_packet) – which encodes that if a device ED finishes a session i by sending an encrypted data-packet p_i to an application server AS , then if the application server AS gets this data-packet p_i it believes it originates from this device ED .
- (7) *two-party confusion-freeness w.r.t. AppSKey* (CF_ED_AS_packet_key) – which encodes the same as CF_ED_AS_packet w.r.t. the AppSKey: i.e., if a device ED finishes a session i with a given AppSKey _{i} , then if the application server AS receives AppSKey _{i} , it correctly believes it is meant for communicating with ED .
- (8) *three-party confusion-freeness w.r.t. AppSKey* (CF_ED_JS_AS_key) – which encodes the same as CF_ED_AS_packet_key, factoring in the JS as well: i.e., the view on a given AppSKey is synchronised correctly between a given ED s, the JS and the AS .

As with LoRaWAN 1.0, we analyse additional lemmas, primarily to check the correctness of our modelling, or for auxiliary security properties (e.g., that the key-setup/commissioning we model between Devices and the Join Server, is secure). However, as they are not pertinent to the analysis, we omit details on those here.

Last but not least, we recall that we run our verification for these properties on both the “Desync-Model”

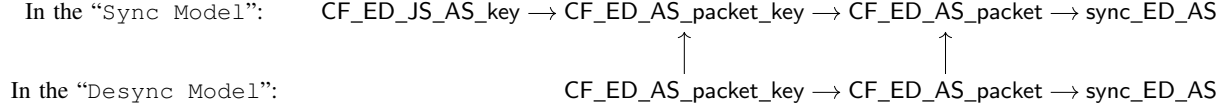


Figure 4: Hierarchy of Confusion-Freeness and Synchronization Goals

and “Sync-Model” of the specifications, in each of the three threat models $\mathcal{M}_{\text{LoRa1.1Spec}}$, $\mathcal{M}_{\text{NS-weakCorrupt}}$, $\mathcal{M}_{\text{AS-NS-Secure}}$, with two different methods for distributing AppSKey. To this end, we created various bespoke oracles for the respective analyses.

4.5. Hierarchy of Properties Analysed

The relationships between our various security properties are as follows:

Agreement Properties. Our agreement properties are directly drawn from the definitions of Lowe [10]. Although the protocols we analyse are not two-party, the agreement results work on a pairwise basis. Hence we have that injective agreement is stronger than non-injective agreement, which in turn is stronger than weak agreement.

Confusion-Freeness Properties. The simplest of the confusion-freeness properties is CF_ED_AS_packet. CF_ED_AS_packet is stronger than sync_ED_AS, as it looks not just at the AS decoding the packets, but also the AS attributing the device address inside said packets to a device. This process is tied in with the format of the application-layer messages per se and (also) with how the device address (that appears in these messages) is fed¹⁰ to the AS. In other words, analysing CF_ED_AS_packet also looks at potential confusion of device identities.

Also, in this vein, CF_ED_AS_packet is weaker than CF_ED_AS_packet_key, as the latter requires agreement on AppSKey as well as the packets sent to be associated with that AppSKey. In fact, note that the confusion-freeness properties (CF_ED_AS_packet_key, CF_ED_JS_AS_key,) that require agreement on both the AppSKeys as well as the data-packet, logically, are not truly needed in the “Desync-Model”: if the AS is able to report failure when decoding the packet, it is less relevant if it receives an incorrect AppSKey. As such, we check the CF_ED_AS_packet_key, CF_ED_JS_AS_key only in the “Sync-Model”.

Some of the confusion-freeness properties require three parties to agree, compared to the simpler case of just two parties. Clearly, the former are stronger requirements than the latter. Not only that, but given the modelling, if the 3 parties at hand agree on an AppSKey (i.e., CF_ED_JS_AS_key holds), then necessarily the AS and ED would agree on the packet as well (i.e., CF_ED_AS_packet_key holds); this is because the first agreement over 3 parties also binding over identification of the device (ED’s EUI and ED’s address), in a way that links in with the packets sent and received by the ED and the AS, respectively.

Other Relationships. Although not necessarily true in the universal setting, an attack on key secrecy for the LoRaWAN Join protocol immediately results in a

scenario where the adversary can forge data packets (and the associated MACs). Such an attack would thus entail violations of the majority of the other properties checked.

We note that the “Desync Model” encodes a weaker system than the “Sync Model”. So, for the formulae that make sense to be checked in both (e.g., CF_ED_AS_packet and sync_ED_AS), we have an implication from “Desync”-based formulations to “Sync”-based formulations.

Unlike in the case of “Desync” vs. “Sync”, hierarchical lines cannot be drawn in between properties holding in the variations of the models w.r.t. AppSKey deliveries: that is, the “AppSKey-from-NS” and “AppSKey-from-JS” are incomparable with respect to these properties holding:

$$\text{prop. in “AppSKey-from-NS”} \not\Rightarrow \text{prop. in “AppSKey-from-JS”}$$

Finally, if any property holds in the $\mathcal{M}_{\text{NS-weakCorrupt}}$ model, then it will hold in the $\mathcal{M}_{\text{LoRa1.1Spec}}$ model. If any property holds in the $\mathcal{M}_{\text{LoRa1.1Spec}}$ model, then it will hold in the $\mathcal{M}_{\text{AS-NS-Secure}}$ model:

$$\mathcal{M}_{\text{NS-weakCorrupt}} \Rightarrow \mathcal{M}_{\text{LoRa1.1Spec}} \Rightarrow \mathcal{M}_{\text{AS-NS-Secure}}$$

In Figure 4 we present the part of our hierarchy of properties that does not trivially follow from Lowe’s agreement lattice, but is rather dictated by our threat and system models.

4.6. Analysis Results

We verified the above properties in all our threat models introduced in Subsection 3.1: $\mathcal{M}_{\text{AS-NS-Secure}}$, $\mathcal{M}_{\text{LoRa1.1Spec}}$, and $\mathcal{M}_{\text{NS-weakCorrupt}}$.

We first report on the analysis of ED-JS properties. Given the presence of a secure channel between the NS and JS in all but the $\mathcal{M}_{\text{NS-weakCorrupt}}$ model, and the implicit assumption that ED is honest in these security claims, these properties hold in the $\mathcal{M}_{\text{LoRa1.1Spec}}$ and $\mathcal{M}_{\text{AS-NS-Secure}}$ cases. However, these properties do not hold in the $\mathcal{M}_{\text{NS-weakCorrupt}}$ threat model, when the adversary is able to manipulate the (mostly plaintext) messages on this channel.

We now present our verification results for the properties between the ED and the AS. Table 2 contains a simplified view of our results, while Table 3 in Section 6 contains the outcomes of all of the analysis performed. Due to the number of different modelling scenarios, we focus our discussion on a few key cases.

We begin with the scenario which most closely follows the LoRaWAN specification. This is the $\mathcal{M}_{\text{LoRa1.1Spec}}$ threat model, using the Desync-Model encoding. This model is weaker than the Sync-Model, as the AS accepting potentially malformed messages enables some attacks.

10. For details, see e.g., pages 12, 18, 19 of the backend specifications.

	sync_ED_AS	CF_ED_AS_packet_key	CF_ED_JS_AS_key
$\mathcal{M}_{\text{LoRa1.1Spec}}$	fails	fails	fails
$\mathcal{M}_{\text{NS-weakCorrupt}}$	fails	fails	fails
$\mathcal{M}_{\text{AS-NS-Secure}}$	holds	holds	holds

Table 2: Oracle-Automated Verification for the LoRaWAN 1.1 Join w.r.t. *ED* and *AS*: Summative Results

In this setting, our analysis shows that sync_ED_AS fails. Further, confusion-freeness CF_ED_AS_packet_key fails, even in the main variant of the protocol where the *AS* obtains the AppSKey from the *NS* (see Subsection 2.1). In this case, our falsifying traces shows that the attacker can cross-wire device addresses and the corresponding AppSKeys as well inject wrong application data-packets.

The second row of Table 2 shows the $\mathcal{M}_{\text{NS-weakCorrupt}}$ model, in which the channel between *NS* and *JS* is considered to be insecure. In this case, the aforementioned properties (sync_ED_AS, CF_ED_AS_packet_key) continue to fail. Moreover, a stronger 3-party variation of the confusion-freeness property, CF_ED_JS_AS_key, fails (also due to the hierarchical implication). More specifically, CF_ED_JS_AS_key fails in the $\mathcal{M}_{\text{NS-weakCorrupt}}$ model because the attacker can confuse not only the *AS*, but also the *JS* as to which devices sent which JoinRequests or messages. Concretely, the trace found here shows that an attacker who controls the channel between *NS* and *JS* never forwards anything to the *JS*, and swaps two devices’ requests on the *NS-AS* channel.

Finally, the third row of Table 2 shows that if all channels are secure (i.e., in the $\mathcal{M}_{\text{AS-NS-Secure}}$), all these 2-party and 3-party properties hold.

4.7. Implications of the Attacks Found

The trace corresponding to CF_ED_AS_packet_key failing in the *Desync Model* shows that *AS* can be (adversarially) confused when considering from which devices application-layer messages have come. As a result, *AS* may be ultimately unable to decode a message from some *ED*₁, because they have received some AppSKey₂ to link to *ED*₁ when this key instead corresponds to some *ED*₂.

It is not clear how the *AS* would behave in this case, or if it would store the wrong address for *ED*₁ for a period of time.

With sync_ED_AS failing, and with the specs not being clear on how the *AS* needs to respond, it also means that an *ED* may well believe it is sending LoRa data to an *AS*, when it actually is not. In fact, a malicious party in between the *NS* and *AS* gets hold of this (encrypted) data. This creates not only data loss for the *ED*’s owner, but also data-collection by malicious parties, with no hope that the *AS* would track or signal any malfunction.

Countermeasures. To stop the failure of sync_ED_AS and CF_ED_AS_packet_key, we first and foremost recommend that the LoRaWAN 1.1 explicitly require that the *AS* check the format of the first data-packet served and end in “error” if the decryption of this eventually fails. Moreover, it should not store any long-term data with respect to this failed message.

Furthermore, we observe that the AKE properties that fail in the $\mathcal{M}_{\text{LoRa1.1Spec}}$ model hold in the model $\mathcal{M}_{\text{AS-NS-Secure}}$. In practice, this means that strengthening the LoRaWAN specification’s requirements to have a secure channel between the *NS* and the *AS* can fix the aforementioned AKE/LoRa 1.1 failing. However, the security of this channel may often fall under the management of the *NS* and *AS*. This means that even if the LoRaWAN specifications were to require that these channels be secure, we cannot be sure that this would be attained in practice.

Our analysis in the $\mathcal{M}_{\text{NS-weakCorrupt}}$ setting tells us that any loosening of the trust in *NS* results in several key-agreement properties failing. Notably, because confusion freeness fails, it means that we cannot guarantee that an *AS* will have a correct view as to which device it is speaking to. At the application-level, this is a significant attack.

Due to the aforementioned issues with security ownership and risk, a more costly countermeasure that may prevent the failure of sync_ED_AS and CF_ED_AS_packet_key is the possible introduction of a clear key-confirmation step between the *AS* and *ED*. Or, even further, one can consider making the *AS* part of the Join and not having the AppSKey delivered “blindly” to it alongside the first data-packet

Overall, it maybe be advisable that the Join Procedure be redesigned such that the *AS* cannot be confused irrespective of the security of these channels and trust in *NS*. However, it is important that any modifications should remain as close as possible to the current Join design.

5. A New, Secure LoRaWAN 1.1 Join

We now propose an amended version of the LoRaWAN 1.1 Join, which we call *LoRa 3-AKA*⁺.

This new design is rooted in the analysis results presented in Subsection 4.6.

In order to prevent the *AS* from becoming confused as to which device it is communicating with, the *NS* provisioner must ensure the security of the incoming and outgoing channels. However, it would be interesting if we could attain the same level of security by instead placing less trust in the messages forwarded by the Network Server. Indeed, it would be ideal if we could propose a design that is secure even when these NS-originating channels are untrusted or compromised. To this end, we leverage that –unlike in the LoRaWAN 1.1 Join– the *AS* could be an active part of the Join. Precisely, in *LoRa 3-AKA*⁺, we augment the LoRaWAN 1.1 Join such that the *AS* is minimally active. Arguably, this is as backwards-compatible as possible with the current LoRaWAN 1.1 Join.

So, at the cost of a small increase in latency, we attain the *LoRa 3-AKA*⁺ Join, which is a secure multi-

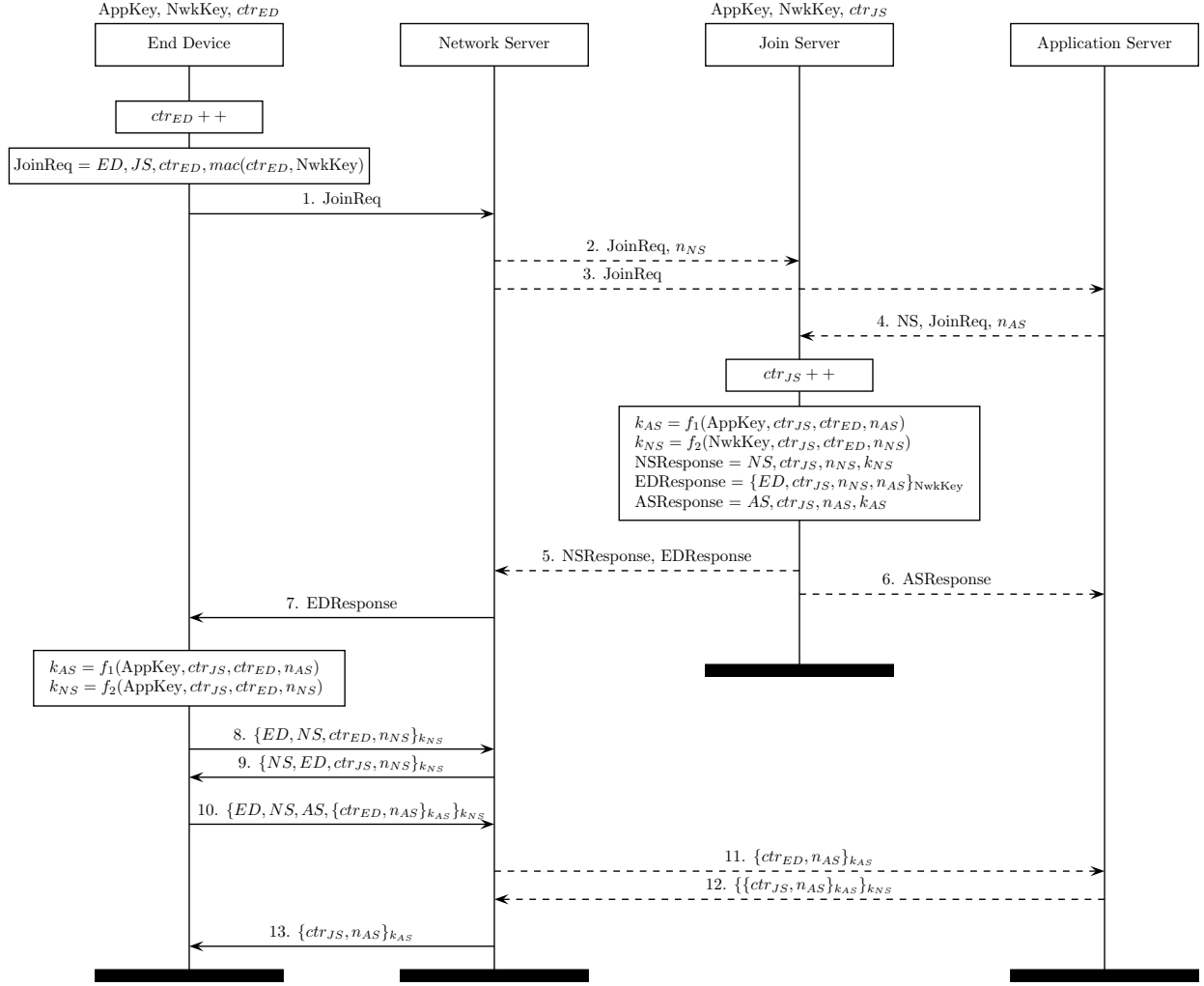


Figure 5: LoRA 3-AKA⁺ – A Minimally-Augmented LoRaWAN 1.1. Join with More Security, Less Trust. Secure channels are displayed with dashed lines “--→”.

party AKE, even when the channels out of the NS are compromised.

Essential Features of LoRA 3-AKA⁺ Compared to the LoRaWAN 1.1 Join. We work with the following principles in mind:

- The parties and network topology remain the same as in LoRaWAN 1.1.
- The Application Server is “minimally” active during the Join Procedure: it contributes a nonce to the Join Request, adding randomness to the encryption key AppSKey and the MAC key NwkSKey.
- We envisage that the LoRA 3-AKA⁺ specification require (with a ‘MUST’ keyword) that the NS - AS and NS - JS channels be secure; however, we show that even if these are compromised the (confusion-freeness) security guarantees are not lost.
- A “Key Confirmation” step is added at the end of the protocol, whereby the involved parties check that they have mutually agree on the established session keys. This key confirmation step occurs *before* the record phase begins.

The LoRA 3-AKA⁺ design is as close as possible to LoRaWAN 1.1 Join, whilst giving protection to the Application Server against any possible (key/device) con-

fusion attacks such as those we have highlighted against LoRaWAN 1.1.

The LoRA 3-AKA⁺ Protocol. Figure 5 gives an overview of LoRA 3-AKA⁺.

The protocol LoRA 3-AKA⁺ is split into two main phases. First, the keys are established using a process very similar to LoRaWAN 1.1. Then, a set of “*Key Confirmation*” messages are sent between the End Device and the Network Server and Application Server respectively, in which the newly generated keys are checked for authenticity.

For readability, in this description we do not include references to parties’ names or configuration parameters. We also collapse the multiple keys NwkSKey, FNwkSIntKey, SNwkSIntKey into a single value (k_{NS}). However, our Tamarin models do model the addition of parties’ names and all LoRaWAN keys.

The LoRA 3-AKA⁺ protocol proceeds as follows:

- 1) The End Device ED generates a Join Request in the same way as in LoRaWAN 1.1, incrementing its DevNonce counter and sending it along with the request command, including a MAC of the message using the NwkKey.

- 2–3) The Network Server NS forwards the Join Request to the Join Server, adding a freshly generated nonce $NwkNonce$. The Join Request is also forwarded as-is to the Application Server.
- 4) The Application Server AS acknowledges the Join Request by forwarding another copy of it to the Join Server, adding a freshly generated nonce $AppNonce$.
- 5–7) The Join Server processes the Join Request, generating the following values:

$k_{AS} = f_1(NwkKey, JoinNonce, DevNonce, AppNonce)$
 $k_{NS} = f_2(NwkKey, JoinNonce, DevNonce, NwkNonce)$
 $NSResponse = JoinNonce, NwkNonce, k_{NS}$
 $EDResponse = \{JoinNonce, NwkNonce, AppNonce\}_{NwkKey}$
 $ASResponse = JoinNonce, AppNonce, k_{AS},$

$NSResponse$, $ASResponse$ and $EDResponse$ are then sent to NS , AS and ED (via NS), respectively.

- 8) The End Device re-calculates the keys independently. It then sends the first *key confirmation* request to the Network Server, consisting of the values $DevNonce$ and $NwkNonce$, encrypted with the calculated k_{NS} .
- 9) The Network Server responds with a *key response* message, containing $JoinNonce$ and $NwkNonce$.
- 10–13) The End Device sends a second key-confirmation message, this time to the Application Server (via the Network Server). These messages are encrypted using k_{AS} , making the message-body unreadable to the Network Server. The key-confirmation message contains values $DevNonce$ and $AppNonce$, and the key-confirmation response contains values $JoinNonce$ and $AppNonce$.

5.1. Analysis of the LoRA 3-AKA⁺ Protocol.

We modelled the LoRA 3-AKA⁺ protocol in the $\mathcal{M}_{AS-NS-Secure}$ and $\mathcal{M}_{NS-weakCorrupt}$ models. These models represent the best-case and worst-case scenarios, depending on whether or not the NS is part of the fully trusted environment alongside the JS and AS .

We modelled the same main properties as for LoRaWAN 1.1 Join.

- (1) *weak agreement between ED and AS* (wa_ED_AS)
- (2) *two-party confusion-freeness* (CF_ED_AS_packet_key)
- (3) *explicit three-party confusion-freeness* (CF_ED_JS_AS_key)

Our analysis inside Tamarin proves that wa_ED_AS , $CF_ED_AS_packet_key$, $CF_ED_JS_AS_key$, all hold on LoRA 3-AKA⁺, in both the $\mathcal{M}_{AS-NS-Secure}$ and $\mathcal{M}_{NS-weakCorrupt}$ threat models. In particular, even when the channels in and out of the NS were compromised, the fact that the AS is alive, as well as the addition of the key confirmation step, prevents any agreement insecurities inside LoRA 3-AKA⁺.

Intuitively, although a corrupt NS is able to grant the adversary access to the secure channels and thus read, redirect or inject messages, the addition of explicit checks prevents risk of confusion. Since the confirmation check between the ED and AS is encrypted using the just-established key (which the adversary cannot reproduce

without knowledge of the long term $AppKey$), agreement and aliveness is ensured.

6. Verification Details

In this section, we join together our security analysis results, and present certain details of the verification process of the LoRaWAN 1.0, LoRaWAN 1.1 Join and the LoRA 3-AKA⁺ Join protocol that we eluded to in Section 4 and Section 5.

Models. We produced 15 Tamarin models altogether: one for LoRaWAN 1.0, twelve for the different LoRaWAN 1.1 scenarios and two for LoRA 3-AKA⁺. Most Tamarin models contain around 1300 lines of code.

We also recall here our different threat models:

- in $\mathcal{M}_{LoRa1.1Spec}$, the channel between the NS and the JS is secure, but the channel between the NS and AS is insecure.
- in $\mathcal{M}_{AS-NS-Secure}$, the channel between the NS and the JS is secure, and the channel between the NS and AS is secure (i.e., it tightens the $\mathcal{M}_{LoRa1.1Spec}$ threat-model).
- in $\mathcal{M}_{NS-weakCorrupt}$, all channels in and out of the NS are insecure but the NS does follow the protocol (i.e., it moderately loosens the level of trust that the LoRaWAN specification places on the NS).

Due to this faithfulness and thus the resulting complexity of our modelling (see Sections 4 and Section 5), the standard heuristics employed by the Tamarin prover to prove the models' properties do not reliably result in termination. Consequently, the proofs of all properties required the assistance of a *proof oracle* (see Section 2, on Tamarin oracles). These oracles were constructed by using the web interface of the Tamarin prover and stepping through the proofs of the desired property. The steps taken in this way, for each model, were then encoded into a corresponding Python script of the order of between 100 and 150 lines of code. Using these oracles, our lemmas are then proved automatically, without any further intervention.

All the models as well as any oracles, and the saved versions of any manual proofs are available at [16], together with instructions on how to run them.

Summary of Results. Table 3, Table 4 and Table 5 summarise all the results we have discussed so far (and a few more), for LoRaWAN 1.0, LoRaWAN 1.1 and LoRA 3-AKA⁺, respectively. A $\checkmark$ in the table indicates that the property holds in that model, a $\times$ indicates that the TAMARIN PROVER found a trace that violated the property. Further, an "NA" entry indicates that the property does not apply to that model, while "NM" indicates that the property was not modelled. Some of the security goals presented in Subsection 4.4 are omitted, because their validity is immediately deducible from the results of other properties, following from our hierarchy of goals presented in Subsection 4.5. Finally, "NT" means that the proof did not terminate given our current oracles. It is possible that with re-working of the associated oracle these would terminate, but this is uncertain due to the underlying undecidability of the verification problem at hand.

Threat Model	$\mathcal{M}_{AS-NS-Secure}$				$\mathcal{M}_{LoRa1.1Spec}$				$\mathcal{M}_{NS-weakCorrupt}$			
Synchronisation	Sync		Desync		Sync		Desync		Sync		Desync	
Key Delivery	NS key deliv.	JS key deliv.	NS key deliv.	JS key deliv.	NS key deliv.	JS key deliv.	NS key deliv.	JS key deliv.	NS key deliv.	JS key deliv.	NS key deliv.	JS key deliv.
Security Goal												
key_secrecy	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
wa_ED_JS	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
wa_ED_AS	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗
nia_ED_JS	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗
sync_ED_AS	✓	✓	✓	✓	✓	✓	✗	✗	NM	NM	✗	✗
CF_ED_AS_packet_key	✓	✓	✓	✓	NT	NT	✗	✗	✗	✗	✓	✓

Table 3: Main Verification Results for the LoRa 1.1 Join

Security Goal	$\mathcal{M}_{LoRaSpec}$
key_secrecy	✓
sync_ED_NS	✗
wa_ED_NS	✓
nia_ED_NS	✗
ia_ED_NS	✗
replay resistance	✗

Table 4: LoRa 1.0 Results

Threat Model	$\mathcal{M}_{AS-NS-Secure}$	$\mathcal{M}_{LoRa1.1Spec}$	$\mathcal{M}_{NS-weakCorrupt}$
Security Goal			
key_secrecy	✓	NA	✓
wa_ED_JS	✓	NA	✓
wa_ED_AS	✓	NA	✓
nia_ED_JS	✓	NA	✓
ia_ED_JS	✓	NA	✓
sync_ED_AS	✓	NA	✓
CF_ED_AS_packet_key	✓	NA	✓

Table 5: Main Verification Results for LoRa 3-AKA⁺

Concerning LoRaWAN 1.0, we recount the main results:

- 1) We re-prove that synchronisation between *ED* and *NS* (i.e., the property defined as `sync_ED_NS` in Section 4) fails. This was known from arguably sketchy modelling of LoRaWAN1.0 in [4];
- 2) We show non-injective agreement and injective agreement between *ED* and *JS* also fails;
- 3) By modelling the small domain of nonces, we show that replay resistance fails. This is the first symbolic-verification analysis finding this attack, known since [1].

For LoRaWAN 1.1, we re-summarise the main results:

- 1) In the $\mathcal{M}_{LoRa1.1Spec}$ threat-profile, we have the following main results:
 - a) using our “Desync Model”, in the case where the *NS* delivers the AppSKey to the *AS*, as well as in the case where the *JS* does so, we formally prove that the *AS* and the *ED* can be desynchronised (i.e., `sync_ED_AS` and `CF_ED_AS_packet_key` failing). The first trace found (on `sync_ED_AS` failing) shows that that *AS* cannot know where a packet originates from. The second trace found is synonymous: there are two genuine device addresses with two AppSKeys and two encrypted data packets present in the system, and the attacker inflicts a message onto *AS* with the first device address, the second AppSKey and a fraudulent data packet, yet the *AS* “throws” no error.
- 2) In the $\mathcal{M}_{AS-NS-Secure}$ threat-profile, we prove that the insecurity problems found in $\mathcal{M}_{LoRa1.1Spec}$ threat-profile disappear.
- 3) In the $\mathcal{M}_{NS-weakCorrupt}$ threat-profile then we show that several agreement properties fail, as expected.
- 4) For LoRa 3-AKA⁺, we show that not only it is AKE-secure in the $\mathcal{M}_{LoRa1.1Spec}$ model, but it maintains AKE-security even in the 4-party sense (i.e., w.r.t. `CF_ED_JS_AS_key` holding), even in the $\mathcal{M}_{NS-weakCorrupt}$ model.

Other Statistics: Timing, Proof-steps. All models were executed using the Tamarin prover running on a server with 2 Intel Xeon E5-2667 CPUs (16 cores, 32 threads) and 378GB of RAM.

The average time over all automatic proofs in the $\mathcal{M}_{LoRa1.1Spec}$ model, which is the one capturing the LoRaWAN specifications as closely as possible, is around 10 minutes using 10 cores.

7. Engagement with the LoRa Alliance

As part of the research process, we engaged with the LoRa Alliance. As well as performing responsible disclosure of our findings, we also held a series of interactions with respect to the wider scope of formally verifying the current, as well as future, LoRaWAN specifications.

In relation our findings, the Security Working Group (SWG) and the Technical Committee (TC) of the LoRa Alliance are in alignment with us regarding the following main aspects:

- 1) clear requirements ought to be stipulated with respect to the security (i.e., confidentiality and integrity) of the NS-AS channel;
- 2) the *AS* should, to some extent, become an active part of Join, as this would increase the capabilities of the *AS* to verify the authenticity of session keys and application-layer messages.

During this interaction, we confirmed that the LoRaWAN specifications do not explicitly require that the *NS-AS* channel be secure. In general, requirements on the *NS-AS* channel and the *AS* itself are slightly under-specified in the current version of the specifications due to the fact that the *NS* and the *AS* have been initially thought of as co-located parties. As the commissioning of proprietary Application Servers becomes increasingly common, the LoRa Alliance acknowledges that the issues of the under-specification of the security of the *NS-AS* channel as well the amount of trust placed on the *NS*

have become more acute. This confirms the validity of our $\mathcal{M}_{\text{LoRa1.1Spec}}$ and $\mathcal{M}_{\text{NS-weakCorrupt}}$ models.

Indeed, within the LoRa Alliance SWG, work is already ongoing to properly specify requirements on the *NS-AS* channel. With regards to the trust placed in the *NS*, it is to be noted that parts of our LoRa 3-AKA^+ design were generally welcome. Concretely, the *AS* contributing with a nonce to the Join procedure is being considered by the LoRa Alliance SWG as a possible solution going forward. However, the key-confirmation step is viewed as potentially too computationally demanding on the end devices; as such, the latter could be simplified and/or made optional, if ever to be adopted. Along these lines, we will be working with the LoRa Alliance SWG and the LoRa Alliance TC (of which we have become a part), to formally verify iterations of new specifications of the LoRaWAN Join that fit our recommendations and are agreeable with other requirements (e.g., *AS*-related connectivity, energy-consumption of the end devices, etc.).

Last but not least, we note that during our latest discussion with the LoRa Alliance (which occurred after the acceptance for publication of this manuscript), it transpired that there is a discrepancy between the LoRaWAN 1.1 specifications and our models. We believe this gap is worth mentioning; this is as follows. In the specifications, it is the *NS* which generates the DevAddr used by the *ED* in the application-layer packets, whereas in our models¹¹, the *JS* generates the DevAddr and the consequent mapping between DevEUIs and DevAddrs. For future work that extends our models in this setting, new properties hinging more strongly on agreement properties with the *NS* would arguably require verification. Also, more generally, note that the trust on the *NS* would be increased in this setting, and as such our $\mathcal{M}_{\text{NS-weakCorrupt}}$ model may require rethinking.

8. Conclusions

Here we briefly summarise our results and discuss options for future avenues of research.

Main Recommendations for LoRaWAN 1.0. Although this is linked to an older recommendation from [1], our results *formally* prove that the domain of the nonces in 1.0 ought to be increased. The LoRa Alliance looked to repair the replay resistance failure by creating a counter-based mechanism for nonces. Whilst we do not prove a direct attack on this, we believe that a better approach would be to increase the domain of the nonces, which would prevent the replay resistance failure, without introducing additional risks.

Main Recommendations for LoRaWAN 1.1. Firstly, our attacks show that the LoRa Alliance should explicitly demand the *NS* and *AS* channel be secured. That is, for instance, the trace that shows CF_ED_AS_packet_key fail in the “Desync” model would not be possible if the attacker could not inflict his own data-packet in between the *NS* and *AS*, or swap the device address in that same message.

11. The confusion on our part stems from the fact that in the 1.1 specification the actual “*JS*” and “*NS*” words are often confused, due to incremental modifications from LoRaWAN 1.0 specifications to LoRaWAN 1.1 specifications – in which the *JS* is introduced.

Secondly, linked to the above, we believe that a tightening of the specification around how the *AS* is to handle errors upon failure to decode a message is necessary. As per the above, confusion attacks were shown to be possible (i.e. when *AS* is served a device address and an AppSKey that do not match). If a packet does not decode correctly, it is not clear how the *AS* should continue.

Thirdly, we show that relying on one single crux of trust (i.e., the current *NS* in LoRaWAN 1.1) is dangerous. Based on the current LoRaWAN specifications, if only the *NS*-to-*JS* channel becomes compromised, then we show that several AKE security goals can fail.

Results Discussion. We have constructed multiple models of the LoRaWAN Join procedure. We have shown that the LoRaWAN 1.1 Join protocol, while presenting improvements over 1.02, is still secure only under an adversary model weaker than that given in the specifications. We introduced a new security goal, described as *confusion freeness*, which models a form of indirect agreement between the End Device and Application Server.

The core of these problems arises from two issues with the LoRaWAN join procedure. First, the Network Server, which acts as a forwarding agent, is implicitly trusted. Second, the Application Server is not active during the Join procedure. Even with the Join Server being a root of trust, the Application Server must also have confidence that *NS* correctly forwards the results of the procedure. To remedy this, we proposed and verified a new protocol, LoRa 3-AKA^+ , which addresses these concerns, and satisfies security goals even against our most powerful adversary.

As we continue to engage with the LoRa Alliance, at <http://loraverif.github.io>, we will maintain a longer version of this paper that contains further details, and a log of updates to our Tamarin models.

Future Work. Even if this is not in the LoRaWAN specifications, we believe strong device authentication and device unlinkability should be studied. The risk of version-downgrade attacks potentially also exists in LoRaWAN. Modelling such attacks may be useful to get a more complete view of the LoRaWAN ecosystem.

Acknowledgements: The research was partly funded by the following EPSRC projects: “Improving customer experience while ensuring data privacy for intelligent mobility - EP/N028295/1” and “AutoPaSS: Automatic Verification of Complex Privacy Requirements in Unbounded-Size Secure Systems - EP/S024565/1”

References

- [1] G. Avoine and L. Ferreira. Rescuing LoRaWAN 1.0. In *IACR Cryptology ePrint Archive*, 2017.
- [2] K. Bhargavan, I. Boureanu, A. Delignat-Lavaud, P. Fouque, and C. Onete. A formal treatment of accountable proxying over TLS. In *2018 IEEE Symposium on Security and Privacy, SP 2018, Proceedings, 21-23 May 2018, San Francisco, California, USA*, pages 799–816, 2018.
- [3] B. Blanchet. Security protocol verification: Symbolic and computational models. In P. Degano and J. D. Guttman, editors, *Principles of Security and Trust*, pages 3–29, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [4] I. Butun, N. Pereira, and M. Gidlund. Analysis of LoRaWAN v1.1 security: research paper. pages 1–6, 06 2018.

- [5] S. Canard and L. Ferreira. Extended 3-Party ACCE and Application to LoRaWAN 1.1. In J. Buchmann, A. Nitaj, and T. Rachidi, editors, *Progress in Cryptology – AFRICACRYPT 2019*, pages 21–38, Cham, 2019. Springer International Publishing.
- [6] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe. A comprehensive symbolic analysis of TLS 1.3. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, pages 1773–1788, New York, NY, USA, 2017. ACM.
- [7] D. Dolev and A. Yao. On the Security of Public-Key Protocols. *IEEE Transactionson Information Theory* 29, 29(2):198–208, 1983.
- [8] N. Durgin, P. Lincoln, J. Mitchell, and A. Scedrov. Undecidability of Bounded Security Protocols. In E. Clarke and N. Heintze, editors, *In Proceedings of Workshop on Formal Methods and Security Protocols (FMSP'99)*, Trento, Italy, 1999. ACM New York.
- [9] M. Eldefrawy, I. Butun, N. Pereira, and M. Gidlund. Formal security analysis of LoRaWAN. *Computer Networks*, 148:328 – 339, 2019.
- [10] G. Lowe. A Hierarchy of Authentication Specifications. In S. Foley and J. Millen, editors, *In Proceedings of the 10th IEEE workshop on Computer Security Foundations (CSFW'97)*, pages 31–43, Massachusetts, US, 1997. IEEE Computer Society.
- [11] G. Lowe. A hierarchy of authentication specifications. pages 31–43. IEEE Computer Society Press, 1997.
- [12] S. Meier, B. Schmidt, C. Cremers, and D. Basin. The TAMARIN Prover for the symbolic analysis of security protocols. In N. Sharygina and H. Veith, editors, *Computer Aided Verification*, pages 696–701, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.
- [13] N. Sornin. LoRaWAN 1.1 Specification (April 2018). *LoRa Alliance*, https://lora-alliance.org/sites/default/files/2018-04/lorawan1m_specification_v1.1.pdf, 1.
- [14] N. Sornin, M. Luis, T. Eirich, T. Kramp, and O. Hersent. LoRaWAN Specification, Version V1.0.2;. *LoRa Alliance*, <https://lora-alliance.org/resource-hub/lorawanr-specification-v102>.
- [15] The Tamarin Team. Tamarin prover manual, 2016. <https://tamarin-prover.github.io/manual/tex/tamarin-manual.pdf> [Online: accessed 09-April-2019].
- [16] S. Wesemeyer, I. Boureanu, Z. Smith, and H. Treharne. LoRaWAN models. GitHub, 2020. <https://github.com/lorawan-tamarin-analysis/lorawan-tamarin-analysis>.

Appendix A.

Tamarin Model Snippets

We give code snippets from our Tamarin prover implementations. Our full source code is available at <https://github.com/lorawan-tamarin-analysis/lorawan-tamarin-analysis>.

LoRaWAN 1.0 Nonce Selection shows our approach to generating nonces with the potential to repeat (due to the short length) within the Join procedure.

Confusion Freeness shows our definition of the Confusion Freeness security property, a specialised indirect agreement property.

Rule Example shows an example of one of our rules. These rules capture in-depth details, modelling each of the individual keys generated and referencing specific lines of the LoRaWAN specification used in creating our model.

Modelling Configurations shows how we use simple rules, early on in the specification, to easily modify our different sets of assumptions, such as the use of secure channels or how the Application Server receives AppSKey.

Appendix B.

Expanded Diagrams

We include some enlarged versions of the message sequence diagrams included within the paper. We also include additional details, such as agent names included in messages. Such terms have little impact on our security analysis, but more closely represent a real implementation. Figures 10 and 11 contain the LoRaWAN 1.1 protocol, showing the two different methods of AppSKeydelivery. In all diagrams, secure channels are displayed with dashed lines $--\rightarrow$.

Figure 6: LoRaWAN 1.0 Parameters Example

```
rule Device_Receive_JoinAccept:
let
  NS='NetworkServer'
  //inputs - ED_Store_02
  DevEUI=ClientID(~random64)

  //inputs - ans
  decoded=sdec(ans,AppKey)
  AppNonce=fst(decoded)
  NetID=fst(snd(decoded))
  DevAddr=fst(snd(snd(decoded)))
  opt_params=fst(snd(snd(snd(decoded))))
  tau_s=snd(snd(snd(snd(decoded))))

  //compute session keys
  pad16='pad_with_0s'
  NwkSkey=senc(<'0x01', AppNonce, NetID, DevNonce, pad16>, AppKey)
  AppSkey=senc(<'0x02', AppNonce, NetID, DevNonce, pad16>, AppKey)
```

Figure 7: LoRaWAN 1.0 Nonce Selection

```
rule EndDevice_GetRootKey:
let
  AppKey=AesKey(~rootAppKey)
  DevRnd='D1'+ 'D2'+ 'D3' // Define all possible values
in
[
  ProvisionAppKey(<DevEUI, AppEUI, AppKey>
    , ED_Store_00(DevEUI, AppEUI)
  ]
--[
  StoredRootKey(DevEUI)
]->
[
  !ED_Store_01(DevEUI, AppEUI, AppKey, DevRnd)
  , ProvideAppKey(DevEUI, AppEUI, AppKey)
  , !Ltk_shared(DevEUI, AppEUI, AppKey)
]

rule Device_Join_Request:
let
  AppKey=AesKey(~rootAppKey)
  DevRnd=rnd1+rnd2+rnd3 // Select one value arbitrarily
  DevNonce=Nonce(rnd1)
  tau_c=MAC(<'MHDR', AppEUI, DevEUI, DevNonce>,AppKey)
  req=<AppEUI, DevEUI, DevNonce, tau_c>
in
[
  !ED_Store_01(DevEUI, AppEUI, AppKey, DevRnd)
]
--[
  DeviceJoinRequest(DevEUI)
  , Running(DevEUI, 'NetworkServer', DevNonce)
  , Role('Device')
  , OnlyOnce('Device_Request')
]->
[
  Out(<'Join_Request', 'NetworkServer', req>
    , ED_Store_02(DevEUI, AppEUI, DevNonce, AppKey)
  )
]
```

Figure 8: Confusion Freeness

```

lemma AS_confusion_freeness:
"
All DevEUIa AppSvr AppSKey1 #i.
  (
    Running_Confree('Join_AS_ED_confree', AppSvr, DevEUIa, AppSKey1) @ i & Role('AppServer') @ i
  )
==>
  (
    Ex JoinEUI #p #q .
      Commit_Confree('Join_ED_JS_confree', DevEUIa, JoinEUI, AppSKey1) @ p &
      Commit_Confree('Join_ED_AS_confree', DevEUIa, JoinEUI, AppSKey1) @ q &
      Role('JoinServer') @ p & Role('EndDevice') @ q
  ) | (
    Ex RevealEvent Entity #krl .
      KeyReveal(RevealEvent, Entity) @ krl &
      Honest(Entity) @ i
  )
"

\end{mylisting}
\end{figure*}

\begin{figure*}
\begin{mylisting}[enhanced]{Modelling Configurations}
rule ChanOut_S [colour=ffffff]:
  [ Out_S( $A, $B, x ), !F_Paired( $A, $B ) ]
  --[ ChanOut_S( $A, $B, x ) ]->
  [ Sec( $A, $B, x ) ]

rule ChanIn_S [colour=ffffff]:
  [ Sec( $A, $B, x ) ]
  --[ ChanIn_S( $A, $B, x ) ]->
  [ In_S( $A, $B, x ) ]

/*
Determine the type by changing this from AStype1 to AStype2 or vice-versa
*/
rule AS_Init:
  let
    AStype1='GetKeyFromNS'
    AStype2='GetKeyFromJS'
  in
  [ ]
  --[
    EntityInit('AS', $AS)
    , OnlyOnce('AS_Init')
  ]->
  [
    !ASInitialised($AS)
    , !AS_Type($AS, AStype1)
  ]

```

Figure 9: Rule Example

```

rule JoinServer_Receive_JoinRequest_Generate_Response_with_Key:
let
  NetID='NetID'
  pad16='pad_with_0s'

  ED_Properties=<ctr_ED_in, ctr_JS_in, JoinNonce_in, DevAddr_in, JSIntKey_in, FNwkSIntKey_in,
  SNwkSIntKey_in, NwkSEncKey_in, AppSKey_in>
  req=<$JoinEUI, $DevEUI, Counter(DevNonce),tau_c>
  NwkKey=AesKey('Nwk',~rootNwkKey)
  AppKey=AesKey('App',~rootAppKey)

  tau_c_dash=<'MHDR', $JoinEUI, $DevEUI, DevNonce>

  ctr_JS=ctr_JS_in+'2'
  JoinNonce=Nonce(<$DevEUI, ctr_JS>)
  DevAddr=Nonce(~rnd32DevAddr)

  //Compute Keys (p55, 1. 1601-1606)
  JSIntKey=SessKey(senc(<'0x06', $DevEUI, pad16>, NwkKey))
  FNwkSIntKey=SessKey(senc(<'0x01',JoinNonce, $JoinEUI,DevNonce, pad16>, NwkKey))
  SNwkSIntKey=SessKey(senc(<'0x03',JoinNonce, $JoinEUI,DevNonce, pad16>, NwkKey))
  NwkSEncKey=SessKey(senc(<'0x04',JoinNonce, $JoinEUI,DevNonce, pad16>, NwkKey))
  AppSKey=SessKey(senc(<'0x02', JoinNonce, $JoinEUI, DevNonce, pad16>, AppKey))

  newED_Properties=<DevNonce, ctr_JS, JoinNonce, DevAddr, JSIntKey, FNwkSIntKey, SNwkSIntKey,
  NwkSEncKey, AppSKey>
  tau_s=MAC(<'0xFF', $JoinEUI, DevNonce, 'MHDR', JoinNonce, NetID, DevAddr,'opt_params'>, JSIntKey)

  ans=senc(<JoinNonce, NetID, DevAddr, 'opt_params',tau_s>, NwkKey)
  sessionID=SessionID(~AppSKey_SID)
  encAppSKey=senc(<AppSKey, $JoinEUI, $DevEUI, $AS>, JS_AS_Key)
  EDResp=<'Join_Accept', $DevEUI, ans>
  NSResp=<'SessionKeys', $DevEUI, $NS, FNwkSIntKey, SNwkSIntKey, NwkSEncKey>
  ASDetails=<'EncAppSKey',encAppSKey>
in
[
  In(<$NS, $JoinEUI, 'Join_Request_Forward', req>)
  , JS_State_01($JoinEUI, $DevEUI, $NS, $AS, ED_Properties)
  , !Ltk_shared($JoinEUI, $DevEUI, NwkKey)
  , !Ltk_shared($JoinEUI, $DevEUI, AppKey)
  , !Ltk_shared($JoinEUI, <$AS, $DevEUI>, JS_AS_Key)
  , !AS_Type($AS,'GetKeyFromNS')
  , Fr(~rnd32DevAddr)
  , Fr(~AppSKey_SID)
]
--[
  Eq(verifyMAC(tau_c_dash, NwkKey, tau_c), accept)
  , LessThan(ctr_ED_in, DevNonce) //check that the device counter has increased
  , GenerateJoinResponse($JoinEUI, $NS, $DevEUI)
  , JoinResponse($JoinEUI, $DevEUI, DevNonce, ctr_JS)
  , Running($JoinEUI, $DevEUI, <FNwkSIntKey, SNwkSIntKey, NwkSEncKey, AppSKey, JSIntKey>)
  , Commit_Confree('Join_ED_JS_confreen', $DevEUI, $JoinEUI, AppSKey)
  , Honest($JoinEUI)
  , Honest($DevEUI)
  , Role('JoinServer')
]->
[
  Out(<$JoinEUI, $NS, 'Join_Response_FromJS', EDResp, NSResp, ASDetails>)
  , JS_State_01($JoinEUI, $DevEUI, $NS, $AS, newED_Properties)
  , !Ltk_shared($JoinEUI, <$DevEUI, $NS>, FNwkSIntKey)
  , !Ltk_shared($JoinEUI, <$DevEUI, $NS>, SNwkSIntKey)
  , !Ltk_shared($JoinEUI, <$DevEUI, $NS>, NwkSEncKey)
  , !Ltk_shared($JoinEUI, <$DevEUI, $AS>, AppSKey)
]

```

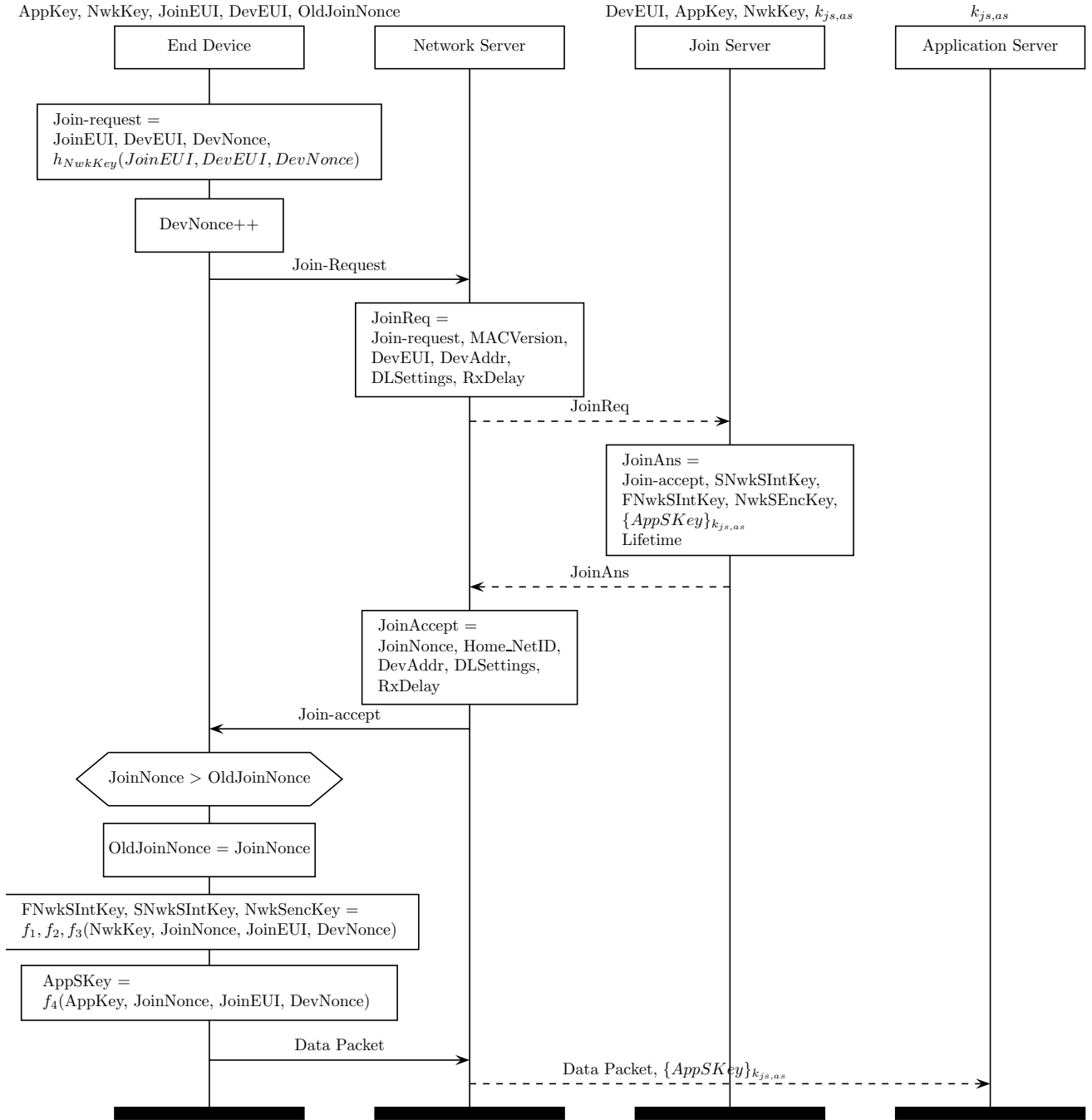


Figure 10: LoRa 1.1, AS Fetches AppSKey from NS

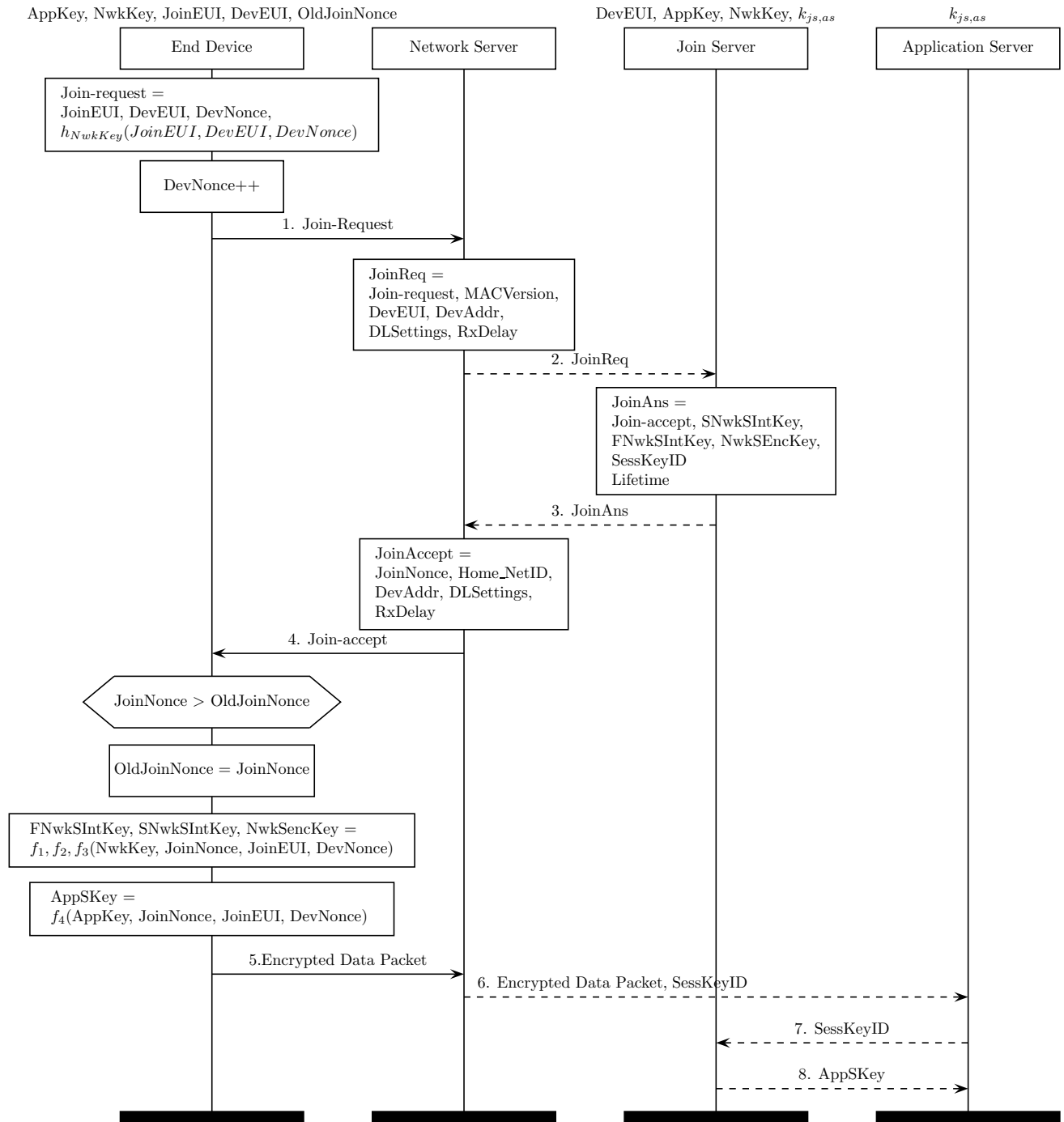


Figure 11: LoRa 1.1, AS Fetches AppSKey from JS